Team Name : The Slaying Ninjas

Team Leader : Padma Priya T R

Team Members : Padmesh G N, Sharan B

Team Mentor : Mrs. V. Thanammal Indu

Problem Statement : Online Fake Content Identification

Dataset : https://www.kaggle.com/ciplab/real-and-fake-face-detection

```
#Importing library
import keras
from keras.models import Sequential
from keras.layers import Dense, Activation, Dropout, Flatten, Conv2D, MaxPooling2D
from tensorflow.keras.layers import BatchNormalization
import numpy as np
```

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

```
################CREATING AND COMPILING ALEXNET MODEL ########################
#Instantiate an empty model
model = Sequential()
# 1st Convolutional Layer
#model.add(Conv2D(filters=96, input_shape=(224,224,3), kernel_size=(11,11), \
#                  strides=(4,4), padding='valid'))
model.add(Conv2D(filters=46, input_shape=(224,224,3), kernel_size=(11,11), \
                  strides=(4,4), padding='valid'))
model.add(Activation('relu'))
# Max Pooling
model.add(MaxPooling2D(pool_size=(2,2), strides=(2,2), padding='valid'))
model.add(BatchNormalization())
# 2nd Convolutional Layer
#model.add(Conv2D(filters=256, kernel_size=(11,11), strides=(1,1), \
#                  padding='valid'))
model.add(Conv2D(filters=123, kernel_size=(11,11), strides=(1,1), \
                  padding='valid'))
model.add(Activation('relu'))
# Max Pooling
model.add(MaxPooling2D(pool_size=(2,2), strides=(2,2), padding='valid'))
model.add(BatchNormalization())
# 3rd Convolutional Layer
#model.add(Conv2D(filters=384, kernel_size=(3,3), strides=(1,1), \
#                  padding='valid'))
model.add(Conv2D(filters=184, kernel_size=(3,3), strides=(1,1), \
                  padding='valid'))
model.add(Activation('relu'))
model.add(BatchNormalization())
# 4th Convolutional Layer
```

```
# 4th Convolutional Layer
#model.add(Conv2D(filters=384, kernel_size=(3,3), strides=(1,1), \
 #                 padding='valid'))
model.add(Conv2D(filters=184, kernel_size=(3,3), strides=(1,1), \
                  padding='valid'))
model.add(Activation('relu'))
model.add(BatchNormalization())
# 5th Convolutional Layer
#model.add(Conv2D(filters=256, kernel_size=(3,3), strides=(1,1), \
#                  padding='valid'))
model.add(Conv2D(filters=123, kernel_size=(3,3), strides=(1,1), \
                  padding='valid'))
model.add(Activation('relu'))
# Max Pooling
model.add(MaxPooling2D(pool_size=(2,2), strides=(2,2), padding='valid'))
model.add(BatchNormalization())
# Passing it to a Fully Connected layer
model.add(Flatten())
# 1st Fully Connected Layer
#model.add(Dense(4096, input_shape=(224*224*3,)))
model.add(Dense(1952, input_shape=(224*224*3,)))
model.add(Activation('relu'))
# Add Dropout to prevent overfitting
model.add(Dropout(0.4))
# 2nd Fully Connected Layer
#model.add(Dense(4096))
model.add(Dense(1952))
model.add(Activation('relu'))
# Add Dropout
model.add(Dropout(0.4))
model.add(BatchNormalization())
# 3rd Fully Connected Layer
#model.add(Dense(1000))
model.add(Dense(600))
model.add(Activation('relu'))

# Add Dropout
model.add(Dropout(0.4))
model.add(BatchNormalization())
# Output Layer
model.add(Dense(2))
model.add(Activation('softmax'))
model.summary()

# Compile the model
model.compile(loss=keras.losses.sparse_categorical_crossentropy, optimizer='adam', me

```

```
 activation_2 (Activation)   (None, 6, 6, 184)         0

 batch_normalization_2 (Batc  (None, 6, 6, 184)        736
 hNormalization)

 conv2d_3 (Conv2D)           (None, 4, 4, 184)         304888

 activation_3 (Activation)   (None, 4, 4, 184)         0
```

```
 batch_normalization_3 (Batc  (None, 4, 4, 184)        736
 hNormalization)

 conv2d_4 (Conv2D)           (None, 2, 2, 123)         203811

 activation_4 (Activation)   (None, 2, 2, 123)         0

 max_pooling2d_2 (MaxPooling  (None, 1, 1, 123)        0
 2D)

 batch_normalization_4 (Batc  (None, 1, 1, 123)        492
 hNormalization)

 flatten (Flatten)           (None, 123)               0

 dense (Dense)               (None, 1952)              242048

 activation_5 (Activation)   (None, 1952)              0

 dropout (Dropout)           (None, 1952)              0

 dense_1 (Dense)             (None, 1952)              3812256

 activation_6 (Activation)   (None, 1952)              0

 dropout_1 (Dropout)         (None, 1952)              0

 batch_normalization_5 (Batc  (None, 1952)             7808
 hNormalization)

 dense_2 (Dense)             (None, 600)               1171800

 activation_7 (Activation)   (None, 600)               0

 dropout_2 (Dropout)         (None, 600)               0

 batch_normalization_6 (Batc  (None, 600)              2400
 hNormalization)

 dense_3 (Dense)             (None, 2)                 1202

 activation_8 (Activation)   (None, 2)                 0

=================================================================
Total params: 6,654,210
Trainable params: 6,647,786
Non-trainable params: 6,424
_________________________________________________________________
```

```
######generating INPUT DATA IMAGES
from keras.preprocessing.image import ImageDataGenerator
# create a data generator
import tensorflow as tf
#datagen =  tf.keras.preprocessing.image.ImageDataGenerator(rotation_range=20,
#   zoom_range=0.15,
#   width_shift_range=0.2,
#   height_shift_range=0.2,
```

```
#   shear_range=0.15,
#   horizontal_flip=True,
#   fill_mode="nearest")
train_datagen = ImageDataGenerator(
        rescale=1./255,
        shear_range=0.2,
        zoom_range=0.2,
        horizontal_flip=True)
# load and iterate training dataset
#datagen.fit(augment=True)
test_datagen = ImageDataGenerator(rescale=1./255)
train_generator= train_datagen.flow_from_directory("/content/drive/MyDrive/real_and_fak
        target_size=(224, 224),
        batch_size=64,
        class_mode='binary')
# load and iterate validation dataset
validation_generator= test_datagen.flow_from_directory(\
      "/content/drive/MyDrive/real_and_fake_face/testing",target_size=(224, 224),
        batch_size=64,
        class_mode='binary')
# load and iterate testing dataset
#test_generator= datagen.flow_from_directory(\
 #     "/content/drive/MyDrive/rice_leaf_diseases/Test",target_size=(224, 224),
  #      batch_size=5,
   #     class_mode='binary')
model.fit_generator(
      train_generator,
      steps_per_epoch=train_generator.samples//64,
      epochs=40,verbose=1,
      shuffle=True,
      validation_data=validation_generator,
      validation_steps=validation_generator.samples //64)
hist=model.evaluate_generator(
      validation_generator,
      steps=10,verbose=1)
print(model.metrics_names)
print(hist)

```

```
Epoch 2/40
25/25 [==============================] - 45s 2s/step - loss: 0.8947 - accuracy: 0
Epoch 3/40
25/25 [==============================] - 45s 2s/step - loss: 0.8216 - accuracy: 0
Epoch 4/40
25/25 [==============================] - 45s 2s/step - loss: 0.7636 - accuracy: 0
Epoch 5/40
25/25 [==============================] - 44s 2s/step - loss: 0.7461 - accuracy: 0
Epoch 6/40
25/25 [==============================] - 44s 2s/step - loss: 0.7496 - accuracy: 0
Epoch 7/40
25/25 [==============================] - 45s 2s/step - loss: 0.7214 - accuracy: 0

Epoch 8/40
25/25 [==============================] - 46s 2s/step - loss: 0.7105 - accuracy: 0
Epoch 9/40
25/25 [==============================] - 46s 2s/step - loss: 0.6932 - accuracy: 0
Epoch 10/40
25/25 [==============================] - 45s 2s/step - loss: 0.7034 - accuracy: 0
```

```
25/25 [==============================] - 45s 2s/step - loss: 0.7034 - accuracy: 0
Epoch 11/40
25/25 [==============================] - 45s 2s/step - loss: 0.7110 - accuracy: 0
Epoch 12/40
25/25 [==============================] - 44s 2s/step - loss: 0.6839 - accuracy: 0
Epoch 13/40
25/25 [==============================] - 45s 2s/step - loss: 0.6788 - accuracy: 0
Epoch 14/40
25/25 [==============================] - 44s 2s/step - loss: 0.6923 - accuracy: 0
Epoch 15/40
25/25 [==============================] - 45s 2s/step - loss: 0.6491 - accuracy: 0
Epoch 16/40
25/25 [==============================] - 45s 2s/step - loss: 0.6575 - accuracy: 0
Epoch 17/40
25/25 [==============================] - 45s 2s/step - loss: 0.6380 - accuracy: 0
Epoch 18/40
25/25 [==============================] - 45s 2s/step - loss: 0.6235 - accuracy: 0
Epoch 19/40
25/25 [==============================] - 44s 2s/step - loss: 0.6168 - accuracy: 0
Epoch 20/40
25/25 [==============================] - 45s 2s/step - loss: 0.6101 - accuracy: 0
Epoch 21/40
25/25 [==============================] - 45s 2s/step - loss: 0.6097 - accuracy: 0
Epoch 22/40
25/25 [==============================] - 44s 2s/step - loss: 0.6097 - accuracy: 0
Epoch 23/40
25/25 [==============================] - 45s 2s/step - loss: 0.5873 - accuracy: 0
Epoch 24/40
25/25 [==============================] - 45s 2s/step - loss: 0.5745 - accuracy: 0
Epoch 25/40
25/25 [==============================] - 45s 2s/step - loss: 0.5702 - accuracy: 0
Epoch 26/40
25/25 [==============================] - 45s 2s/step - loss: 0.5728 - accuracy: 0
Epoch 27/40
25/25 [==============================] - 45s 2s/step - loss: 0.5788 - accuracy: 0
Epoch 28/40
25/25 [==============================] - 45s 2s/step - loss: 0.5649 - accuracy: 0
Epoch 29/40
25/25 [==============================] - 45s 2s/step - loss: 0.5488 - accuracy: 0
Epoch 30/40
```

```
import pickle
with open("/content/drive/MyDrive/Alexnetfake","wb") as f:
  pickle.dump(model,f)
```

```
INFO:tensorflow:Assets written to: ram://ff8197d7-825d-4636-97e6-7a7d8a057d1a/assets
```

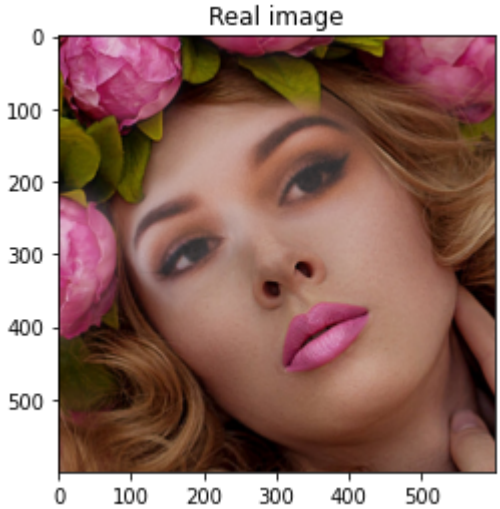
```
import pickle
with open("/content/drive/MyDrive/Alexnetfake","rb") as f:
  al=pickle.load(f)
```

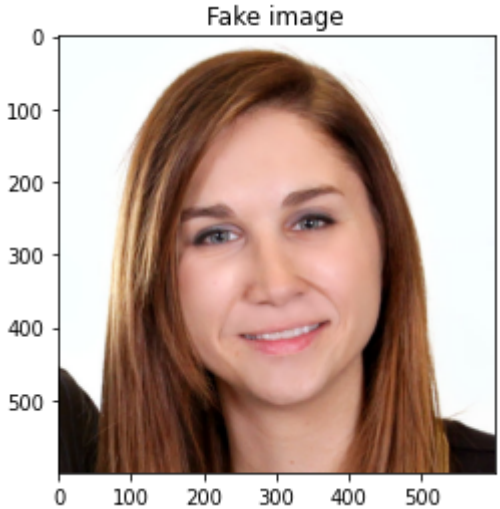
```
test_generator= test_datagen.flow_from_directory(\
      "/content/drive/MyDrive/real_and_fake_face/prediction",target_size=(224, 224),
        batch_size=64,
        class_mode='binary')
```

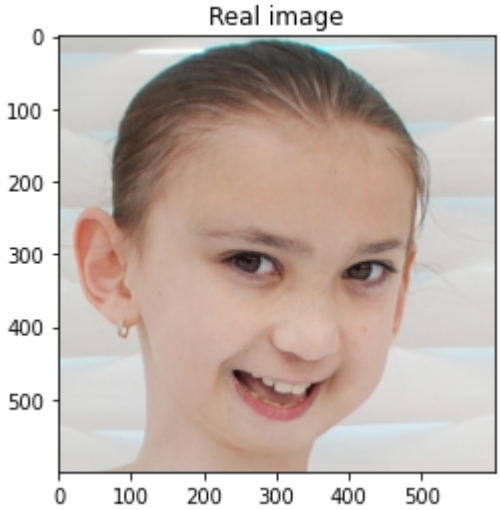
```
pred = al.predict_generator(test_generator, steps=len(test_generator), verbose=0)
# Get classes by max element in np (as a list)
classes = list(np.argmax(pred, axis=1))
# Get filenames (set shuffle=false in generator is important)
filenames = test_generator.filenames
```

```
Found 9 images belonging to 2 classes.
/usr/local/lib/python3.7/dist-packages/ipykernel_launcher.py:4: UserWarning: `Model.
  after removing the cwd from sys.path.
```

```
from tensorflow.keras.preprocessing import image
import matplotlib.pyplot as plt
import numpy as n
from PIL import Image
for i in range(len(filenames)):
  fname="/content/drive/MyDrive/real_and_fake_face/prediction/"+filenames[i]
  img = Image.open(fname)
  plt.figure()
  plt.imshow(img)
  #print(filenames[i])
  if (classes[i]==0):
   # print("Fake")
    plt.title("Fake image")
  else:
   # print("Real")
    plt.title("Real image")
```

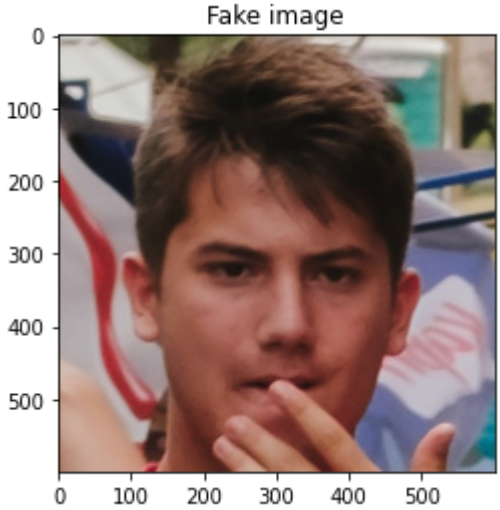

Real image
0
100
200
300
400
500
0
100
200
300
400
500

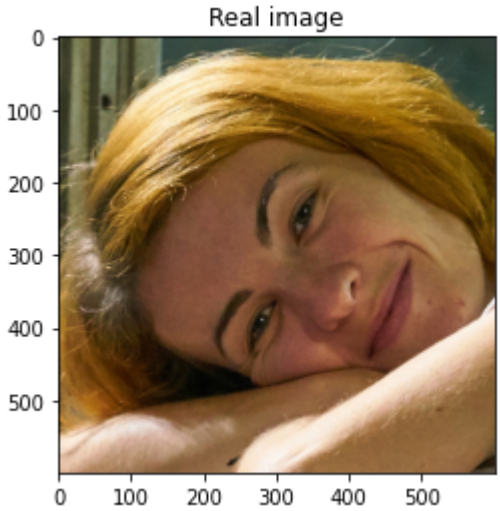

Fake image
0
100
200
300
400
500
0
100
200
300
400
500

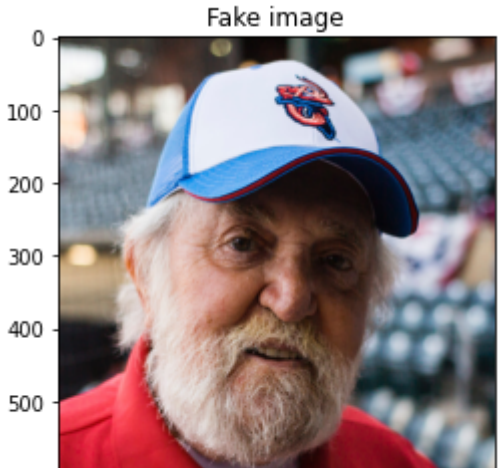

Real image
0
100
200
300
400
500
0
100
200
300
400
500

Fake image
0
100
200
300
400
500

Real image


Fake image


Real image


Fake image

✓ 5s completed at 1:17 PM